PROYECTO 2

ST4CK OVER-PIG

MANUAL TÉCNICO

DAYANA SOFÍA OROZCO MENDÓZA 202231396

ÍNDICE

- Introducción 3
- Tecnologías Utilizadas 4
- Arquitectura Por Paquetes 5
- Diagrama de Clases 6
- Especificaciones de palabras reservadas, símbolos y gramáticaQuinta parte de la propuesta 7
- Tabla de compatibilidad de tipos 12

INTRODUCCIÓN

St4ck Over-Pig es un entorno que permite escribir, validar y compilar programas escritos en tres lenguajes de alto nivel propios:

- Y? (.y) — define estructuras y funciones reutilizables (como una librería).
- Zetariano (.z) — define clases orientadas a objetos.
- Pig Latin (.pig) — es el lenguaje "principal": el punto de entrada del programa, que puede importar las estructuras/funciones de un .y y las clases de un .z.

El programa valida el código de forma léxica, sintáctica y semántica, lo traduce a código de tres direcciones (cuartetas), y finalmente genera y ejecuta código C equivalente — es decir, cubre todo el proceso desde el código fuente hasta un programa funcional.

Requisitos para usarlo:

- Java 21 instalado (o el ejecutable/.jar ya empaquetado con Java incluido, según cómo te lo hayan entregado).
- Un compilador de C instalado y accesible desde la línea de comandos (gcc, clang o cc) — es necesario para que el botón Ejecutar de la pestaña "Código C" funcione.
- Un compilador de Risc-V instalado y accesible desde la línea de comandos (RARS) — es necesario para que el botón Ejecutar de la pestaña "RISC-V" funcione.

TECNOLOGÍAS UTILIZADAS

Tecnología	Versión	Uso en el proyecto
Java	21	Lenguaje de implementación de todo el compilador y la interfaz gráfica.
Maven	-	Gestión de dependencias, build y ciclo de vida del proyecto (pom.xml).
ANTLR4	4.13.2	Generación de los analizadores léxicos y sintácticos (lexer + parser + visitor) de los tres lenguajes fuente (Y?, Zetariano, Pig Latin), a partir de las gramáticas .g4.
JavaFX (controls, fxml, web)	21.0.2	Interfaz gráfica de escritorio: ventana principal, árbol de trabajo, pestañas de editor, panel de resultados/consola.
Maven Shade Plugin	3.5.1	Empaquetado del proyecto en un .jar ejecutable con todas sus dependencias (fat jar).
ANTLR4 Maven Plugin	4.13.2	Integra la generación de código de ANTLR (lexer/parser/visitor) al ciclo de build de Maven, a partir de src/main/antlr4.
JavaFX Maven Plugin	0.0.8	Permite ejecutar la aplicación JavaFX directamente desde Maven.
Compilador de C (gcc / clang / cc)	— (herramienta externa del sistema operativo, detectada en el PATH)	Compila y ejecuta el archivo .c generado a partir de las cuartetas, para producir el bytecode preliminar ejecutable.
Compilador de Risc-V (RARS)	herramienta externa detectada para la ejecución del Risc-v	Compila y ejecuta el archivo .asm generado a partir de las cuartetas, para producir el bytecode preliminar ejecutable.

ARQUITECTURA POR PAQUETES

Paquete	Responsabilidad
parser	Gramáticas .g4 de los tres lenguajes y los AST Builders (PigLatinASTBuilder, YLanguageASTBuilder, ZetarianoASTBuilder) que convierten el árbol de ANTLR en el AST propio del proyecto.
lexer	Lexer personalizado (CustomYLanguageLexer) que envuelve el lexer generado por ANTLR para Y?, agregando el manejo de indentación (tokens INDENT/DEDENT) .
ast	Nodos del árbol de sintaxis abstracta compartido entre los tres lenguajes (VarDeclNode, ParamNode, FunctionDeclNode, ClassDeclNode, etc.).
semantico	Analizador semántico (SemanticAnalyzer), tabla de símbolos, contexto del programa (ProgramContext) y la tabla de compatibilidad/coerción de tipos (TypeCoercionTable).
Traductor	Generación de código intermedio (cuartetas, Quadruple) y generación del código C final (CEmitter).
servicio	Orquestación del proceso de compilación (ServicioCompilacion), manejo de errores de ANTLR (ColectorErroresAntlr), operaciones de archivo y ejecución del C generado (EjecutorC).
resaltado	Resaltado de sintaxis en tiempo real, escrito a mano (tokenizado propio + EstilosRichText).
ui	Ventana principal, árbol de trabajo, editor de archivo, panel de resultados/consola (JavaFX).
util	Utilidades transversales: Type (tipo interno unificado) y TypeMapper (traduce la palabra de tipo de cada lenguaje al tipo interno).

DIAGRAMA DE CLASES



ESPECIFICACIONES DE PALABRAS RESERVADAS, SÍMBOLOS Y GRAMÁTICA

Los tres lenguajes comparten el mismo motor de análisis semántico (un único SemanticAnalyzer y un único Type interno), pero cada uno tiene su propia gramática ANTLR4 independiente (YLanguage.g4, ZetarianoLanguage.g4, PigLatinLanguage.g4, en src/main/antlr4/sofiaO/parser). A continuación el detalle de cada uno.

2.1 Y? (extensión .y)

Lenguaje de indentación significativa (estilo Python), case-sensitive. Solo permite definir estructuras y funciones; no admite variables globales.

Palabras reservadas

Palabra	Significado
%estructuras	Encabezado de la sección de estructuras (opcional).
%funciones	Encabezado de la sección de funciones (obligatoria).
estructura	Declara una estructura.
definir	Declara una función.
->	Indica el tipo de retorno de una función.
retornar	Sentencia de retorno.
entero, flotante, cadena, caracter, bool	Tipos primitivos.
verdadero, falso	Literales booleanos.
imprimir, leer	Entrada/salida.
si, entonces, sino, contrario	Condicional.
elegir, caso, siempre	Switch/case.
para, mientras, hacer	Ciclos (for, while, do).
romper, continuar	Interrupción de ciclos.

Simbolos y operadores

Categoría	Símbolos
Aritméticos	+ - * /
Relacionales	== != < > <= >=
Lógicos	&& || !
Incremento/decremento	++ --
Asignación	=
Estructurales	: , ( ) [ ] { }
Indentación	INDENT / DEDENT (no son caracteres; los genera el lexer personalizado a partir de los espacios/tabs al inicio de línea — ver CustomYLanguageLexer)

Gramática (resumen)

```
programa          : seccionEstructuras? seccionFunciones EOF
seccionEstructuras: '%estructuras' declaracionEstructuras+
declaracionEstructuras: 'estructura' ID ':' atributoEstructura+  (bloque por indentación)
seccionFunciones : '%funciones' declaracionFunciones+
declaracionFunciones: 'definir' ID '(' parametros? ')' ('->' tipo)? ':' bloqueFuncion
```

- Los parámetros se pasan por valor si son primitivos, por referencia si son arreglos ([] tipo ID) o estructuras ({} TipoEstructura ID).
- Los bloques (bloqueFuncion) se delimitan por indentación (INDENT/DEDENT), no por llaves.
- si/entonces/sino/contrario, mientras/hacer, hacer/mientras (do-while) y para siguen esta misma regla de bloques por indentación.
- Gramática completa y comentada: src/main/antlr4/sofiaO/parser/YLanguage.g4.

2.2 Zetariano (extensión .z)

Orientado a objetos, sintaxis inspirada en Java, con llaves y punto y coma. El archivo debe llamarse igual que la clase que define (restricción semántica, validada en ServicioCompilacion, no en la gramática).

Palabras reservadas

Palabra	Significado
public, private, protected	Encapsulamiento para métodos y atibutos de los objetos.
class	Declaración de clase.
extends	Herencia de clases
@Override	Anotación para polimorfismo
void	Tipo de retorno vacío
new	Creación de objetos/arreglos
int/double/char/boolean /string	Tipos primitivos
true/false/null	Literales
if/else	Condicional
switch/case/default	Switch/case
for/while/do	Ciclos
break,continue	Interrupción de ciclos
return	Retorno

Simbolos y operadores

Categoría	Símbolos
Aritméticos	+ - * / %
Relacionales	== != < > <= >=
Lógicos	&& || !
Incremento/decremento	++ --
Asignación	= += -= *=
Estructurales	: , ( ) [ ] { }
Ternario	? :

Gramática (resumen)

```
programa     : 'public' 'class' ID '{' miembroClase* '}'
miembroClase : atributo | constructorDecl | metodoDecl
atributo     : tipo ID ';'
constructorDecl: 'public' ID '(' parametros? ')' bloque
metodoDecl   : 'public' (tipo | 'void') ID '(' parametros? ')' bloque
```

- Los constructores admiten sobrecarga (varias firmas para el mismo nombre de clase); ver ProgramContext.constructoresPorClase.
- Los objetos viven en el heap (se traducen a malloc en el C generado).
- No se contemplan encapsulamiento, herencia ni polimorfismo (quedan pendientes para el Proyecto 2, según el enunciado).
- Precedencia de expresiones de menor a mayor (de afuera hacia adentro en la gramática): ternario → || → && → igualdad → relacional → aditiva → multiplicativa (incluye %) → unaria (!, -) → primaria.
- Gramática completa: src/main/antlr4/sofiaO/parser/ZetarianoLanguage.g4.

2.3 Pig Latin (extensión .pig)

Lenguaje de entrada del programa (el "punto de arranque"). Case-sensitive. No permite definir estructuras ni clases propias: todo tipo no primitivo debe importarse desde un .y o un .z.

Palabras reservadas

Palabra	Significado
import	Importar un archivo .y o .z
VARIABILES>	Encabezado de la sección de variables globales (opcional).
MAIOR>	Encabezado de la función principal (obligatorio).
FINIS	Cierre del programa completo.
finis	Cierre de un bloque si/dum (minúscula: el lenguaje es case-sensitive y distingue ambos cierres).
esto	Declara una variable.
series	Declara un arreglo.
novus	Crea un objeto (equivalente a un new)
si, aliter	Codicional (if/else)
dum	Ciclo tipo while
facere	Ciclo tipo do (se usa con el dum al cierre, formando un do-while)
per	Ciclo tipo for
pergere, interrumpe	continue/break
verum, falsus	Literales booleanos (no llevan palabra de tipo, el tipo se infiere del literal)
numerus, textum, decimaliss, littera	Tipos primitivos (entero, cadenas, flotante, carácter)

Simbolos y operadores

Categoría	Símbolos
Aritméticos	+ - * /
Relacionales	== != < > <= >=
Lógicos	&& || !
Incremento/decremento	++ --
Asignación	=
Entada/salida	>> (imprimir, encadenable), << (leer)
Estructurales	: , ( ) [ ] { }
Comentarios	## ... ## (bloque, se descarta en el lexer)

Gramática (resumen)

```
programa        : importacion* seccionVariables? seccionPrincipal EOF
importacion     : 'import' ID ('.' ID)*
seccionVariables : 'VARIABILES>' declaracionGlobal+
seccionPrincipal : 'MAIOR>' sentencia+ 'FINIS' ';'
```

- Los bloques de si, dum (while) y aliter se cierran explícitamente con finis;; facere { ... } dum(cond); es el ciclo do-while y no lleva finis; per (...) { ... } (for) tampoco lleva finis.
- >> expr >> expr ... ; permite imprimir varios valores encadenados en una sola sentencia.
- Gramática completa, incluyendo un listado de ambigüedades del enunciado original y la decisión tomada para cada una: src/main/antlr4/sofiaO/parser/PigLatinLanguage.g4 (comentarios al final del archivo).

TABLA DE COMPATIBILIDAD DE TIPOS

3.1 Tipo interno unificado

Aunque cada lenguaje usa sus propias palabras clave, el analizador semántico traduce todas a un único enum interno (sofiaO.util.Type), a través de TypeMapper.deTexto(...). Por eso las reglas de compatibilidad/coerción de tipos son exactamente las mismas para los tres lenguajes: se resuelven una sola vez, en TypeCoercionTable, sin importar de qué .y, .z o .pig provenga cada operando.

Tipo interno	Y?	Zetariano	Pig Latin
ENTERO	entero	int	numerus
FLOTANTE	flotante	double	decimalis
CADENA	cadena	String	textum
CARACTER	caracter	char	littera
BOOL	bool	boolean	(sin palabra de tipo: se infiere de verum/falsus)
ESTRUCTURA	tipo definido con estructura	—	tipo importado desde un .y
CLASE	—	tipo definido con class	tipo importado desde un .z
NULO	—	null	—

3.2 Compatibilidad en asignaciones (destino = origen)

Destino\ Origen	ENTERO	FLOTANTE	CADENA	CARACTER	BOOL	ESTRUCTURA/CLASE (mismo tipo)	ESTRUCTURA/CLASE (null)
ENTERO	Si	No	Ni	No	No	No	No
FLOTANTE	Si (conversión implícita)	Si	No	No	No	No	No
CADENA	No	No	Si	No	No	No	No
CARACTER	No	No	No	Si	No	No	No
BOOL	No	No	No	No	Si	No	No
ESTRUCTURA/ CLASE	No	No	No	No	No	Si (mismo tipoUsuario)	Si

La única conversión implícita permitida en una asignación es ENTERO → FLOTANTE (TypeCoercionTable.compatibles). Para estructuras/clases, además del Type debe coincidir el tipoUsuario (el nombre concreto de la estructura/clase); a un struct u objeto sí se le puede asignar null.

3.3 Operador + (suma / concatenación)

LLa regla (TypeCoercionTable.resultadoSuma) es la misma para los tres lenguajes:

Izquierda \ Derecha	ENTERO	FLOTANTE	CADENA	CARACTER	BOOL
ENTERO	ENTERO	FLOTANTE	CADENA	ERROR*	ERROR
FLOTANTE	FLOTANTE	FLOTANTE	CADENA	ERROR*	ERROR
CADENA	CADENA	CADENA	CADENA	CADENA	CADENA
CARACTER	ERROR*	ERROR*	CADENA	CARACTER	ERROR
BOOL	ERROR	ERROR	CADENA	ERROR	ERROR

* ENTERO/FLOTANTE combinados con CARACTER (sin que haya una CADENA de por medio) dan ERROR, porque resultado() no contempla esa combinación aritmética; solo se permite CARACTER + CARACTER. ESTRUCTURA, CLASE, NULO y VOID siempre dan ERROR en cualquier combinación con +.

Ejemplos concretos:

- "Edad: " + 25 → CADENA (concatenación: el entero se convierte a texto).
- 25 + "Edad" → CADENA (mismo resultado sin importar el orden de los operandos).
- 10 + 3.5 → FLOTANTE.
- 10 + 5 → ENTERO.
- "activo: " + verum/true/verdadero → CADENA.
- verum + 1 (booleano + número, sin cadena de por medio) → ERROR.
- Como la evaluación es de izquierda a derecha, "a" + 1 + 2 da "a12", pero 1 + 2 + "a" da "3a" (igual que en Java).

3.4 Operadores aritméticos - * / %

Regla (TypeCoercionTable.resultadoAritmetico): nunca aceptan CADENA ni BOOL, solo tipos numéricos o CARACTER.

Izquierda \ Derecha	ENTERO	FLOTANTE	CARACTER	CADENA / BOOL
ENTERO	ENTERO	FLOTANTE	ERROR	ERROR
FLOTANTE	FLOTANTE	FLOTANTE	ERROR	ERROR
CARACTER	ERROR	ERROR	CARACTER	ERROR

(El operador % solo existe en Zetariano; en Y? y Pig Latin no está definido en la gramática, así que ni siquiera llega a evaluarse esta tabla para esos dos lenguajes.)

3.5 Comparaciones (== != < > <= >=) y lógicos (&& || !)

- Las comparaciones ==/!= entre ESTRUCTURA/CLASE/NULO se resuelven aparte (no en TypeCoercionTable, sino directamente en SemanticAnalyzer.visit(BinaryExprNode)), porque ahí sí hace falta comparar el tipoUsuario concreto y no solo el Type general.
- Los operadores lógicos (&&, ||, !) solo operan sobre BOOL.

3.6 Mapeo final a C (para la generación de código)

Como referencia final de compatibilidad, así traduce Type.aC() cada tipo interno al código C generado (igual para los tres lenguajes, ya que comparten el mismo CEmitter):

Tipo interno	Tipo en C
ENTERO	int
FLOTANTE	double
CADENA	char* (arreglo char[256] cuando es campo de un struct/clase)
CARACTER	char
BOOL	int
ESTRUCTURA / CLASE	NombreDelTipo* (puntero al struct concreto; void* solo si no se conoce el tipoUsuario)
NULO	void*
VOID	void